

S 01. Priprema podataka

U ovom dijelu prikazan je postupak pripreme podataka za klasifikaciju antimikrobnih peptida. Prikazani su koraci učitavanja podataka, provjere njihove kvalitete, čišćenja sekvenci, pripreme negativnog (non-AMP) skupa podataka te izrade konačno uravnoteženog skupa koji će se koristiti u daljnjim fazama projekta.

Cilj ovog dijela projekta je učitati peptidne sekvence iz baze DBAASP, provjeriti njihovu kvalitetu i pripremiti skup podataka za daljnju bioinformatičku analizu. Tijekom priprema podataka uklanjaju se neispravni zapisi, filtriraju monomerni peptidi, provjerava valjanost aminokiselinskih sekvenci te se formira očišćeni skup podataka koji će se koristiti u sljedećim fazama projekta.

1. Uvoz biblioteke

Učitavaju se potrebne biblioteke i pomoćne funkcije koje će se koristiti tijekom pripreme podataka, uključujući učitavanje, provjeru, čišćenje i spremanje skupa podataka.

```
import sys #Uvoz ugrađenog Python modula za rad sa sustavom i putanjama.

sys.path.append( "/content/src" ) # Dodajte mapu /content/src kako bi Pytho

from data_utils import (
    load_dataset,          # učitava dataset u DataFrame
    check_required_columns, # provjerava postoje li svi obavezni stupci
    print_dataset_info,    # ispisuje osnovne informacije u datasetu (b
    check_missing_values,  # provjerava postoje li nedostajuće vrijedno
    filter_monomers,       # zadržava samo ispravne peptide sekvence
    clean_sequences,       # čiste sekvence (uklanja razmake i neisprav
    add_label,             # dodaje oznake klase (npr. 1=AMP, 0=nije AM
    save_processed_dataset # sprema obrađeni dataset za dalju obradu
)
```

2. Učitavanje skupa podataka

Učitava se izvorni skup podataka iz CSV datoteke te se prikazuje prvih nekoliko redaka radi provjere ispravnosti učitavanja.

```
# Putanja do sirovih podataka.
raw_data_path = "/content/peptides.csv"

# Učitavanje podataka.
df = load_dataset(raw_data_path)
```

```
# Prikaz prvih pet redaka.
df.head()
```

	ID	COMPLEXITY	NAME	N TERMINUS	
0	1	Multimer	Distinctin	NaN	NLVSGLIEARKYLEQLHRKLNCKV ENREVP
1	3	Multimer	Halocidin	NaN	WLNALLHHGLNCAKGVLA ALLH
2	4	Multimer	Khal	NaN	KWLNALLHHGLNCAKGVLA ALLH

Opis rezultata

- CSV datoteka uspješno je učitana u Pandas DataFrame (`df`).
- Prikaz prvih pet redaka potvrđuje da su podaci ispravno učitani.
- Skup podataka sadrži informacije o antimikrobnim peptidima, uključujući identifikacijski broj (ID), naziv peptida (NAME), aminokiselinsku sekvencu (SEQUENCE), vrstu sinteze (SYNTHESIS TYPE), ciljnu skupinu mikroorganizama (TARGET GROUP) i ciljni objekt djelovanja (TARGET OBJECT).
- U pojedinim stupcima, poput **N TERMINUS** i **C TERMINUS** , uočene su nedostajuće vrijednosti (NaN), koje će biti obrađene u fazi pripreme i čišćenja podataka.
- Aminokiselinske sekvence uspješno su učitane te su spremne za daljnju analizu i ekstrakciju značajki.

3. Provjera tipova peptida

Analizira se broj zapisa prema tipu peptida kako bi se utvrdilo koliko skup podataka sadrži monomerne, multi-peptidne i multimerne peptide.

```
# Broj zapisa po tipu peptida.
df[ "SLOŽENOST" ].value_counts()
```

	count
COMPLEXITY	
Monomer	1976
Multi-Peptide	21
Multimer	3

dtype: int64

Opis rezultata

Prikaz rezultata pokazuje da izvorni skup podataka sadrži **1976 monomernih peptida** , **21 multi-peptid** i **3 multimerna peptida** .

Iz dobivenih vrijednosti vidljivo je da najveći dio skupa čine monomerni peptidi, dok su multi-peptidi i multimeri zastupljeni u vrlo malom broju. Budući da je cilj projekta klasifikacije monomernih antimikrobnih peptida, u daljnjoj obradi zadržani su samo monomeri.

4. Filtriranje monomernih peptida

U ovom koraku provodi se filtriranje skupa podataka kako bi se zadržali samo monomerni peptidi.

```
# Zadržavanje samo monomernih peptida.
df_monomer = filter_monomeri(df)

# Prikaz dimenzija nakon filtriranja.
ispis (df_monomer.shape)

# Prikaz prvih pet redaka.
df_monomer.head()
```

(1976, 9)

	ID	COMPLEXITY	NAME	N TERMINUS	SEQUENCE	TERM
0	8	Monomer	NaN	C16	KVvWKWvKvVK	
1	10	Monomer	NaN	NaN	LFIFFF	

Opis rezultata

U ovom koraku iz izvornog skupa podataka izdvojeni su samo **monomerni peptidi** , jer su oni predmet daljnje analize i klasifikacije. Filtriranje je provedeno pomoću funkcije `filter_monomers(df)` , koja zadržava samo zapise kod kojih stupac **COMPLEXITY** ima vrijednost **"Monomer"** .

Nakon filtriranja dobiven je skup dimenzija podataka **(1976, 9)** , što znači da sadrži **1976 peptidnih sekvenci** opisanih s **9 atributa** . Prikaz prvih pet redaka potvrđuje da su podaci ispravno filtrirani te da su zadržani svi važni podaci, poput aminokiselinske

sekvence (**SEQUENCE**), vrste sinteze (**SYNTHESIS TYPE**) i ciljne skupine (**TARGET GROUP**).

Ovim postupkom uklonjeni su svi zapisi koji nisu relevantni za projekt, čime je dobiven ujednačen skup podataka pogodan za daljnju obradu i izradu modela strojnog učenja.

5. Čišćenje sekvenci

Uklanjanje se neispravne sekvence, nestandardne aminokiseline i duplicirani zapisi kako bi skup podataka bio prikladan za daljnju obradu.

```
# Čišćenje sekvenci.
df_clean = clean_sequences(df_monomer)

# Ispis dimenzija nakon čišćenja.
ispis (df_clean.shape)

# Prikaz prvih pet redaka.
df_clean.head()
```

(1672, 9)

	ID	COMPLEXITY	NAME	N TERMINUS	SEQUENCE	TERM
0	8	Monomer	NaN	C16	KVVVKWVVKVVK	
1	10	Monomer	NaN	NaN	LFIFFF	
2	11	Monomer	Cathelcidin-1, CATH-1, Fowlicidin-1	NaN	RVKRVWPLVIRTVIAGYNLYRAIKKK	

Opis rezultata

U ovom koraku provodi se čišćenje peptidnih sekvenci kako bi se dobio kvalitetan skup podataka za daljnju analizu i treniranje modela strojnog učenja. Uklonjene su sekvence koje sadrže nestandardne aminokiseline, neispravni zapisi te duplicirane sekvence, odnosno zapisi koji bi mogli negativno utjecati na rezultate klasifikacije.

Nakon čišćenja ispisane su dimenzije novog skupa podataka pomoću naredbe `df_clean.shape`. Dobiven je rezultat **(1672, 9)**, što znači da skup sada sadrži **1672 peptida** opisana pomoću **9 atributa**. U odnosu na prethodni korak, u kojem je bilo **1976 monomernih peptida**, uklonjena su **304 zapisa** koji nisu zadovoljili zadane kriterije kvalitete.

Prikaz prvih pet redaka pomoću naredbe `df_clean.head()` potvrđuje da je skup podataka uspješno očišćen te da su zadržani svi važni atributi koji će se koristiti u nastavku projekta.

Čišćenje podataka provedeno je kako bi model učio samo na kvalitetnim i ispravnim sekvencama. Uklanjanjem nestandardnih aminokiselina, neispravnih i dupliciranih zapisa smanjuje se mogućnost pogrešaka tijekom treniranja te se povećava pouzdanost dobivenih rezultata. Time je dobiven kvalitetniji skup podataka koji je spreman za izdvajanje bioinformatičkih značajki, treniranje modela i njihovu evaluaciju.

6. Dodavanje oznaka klase

Pozitivnim primjerima dodjeljuje se oznaka klase (`label = 1`) koja će se koristiti tijekom treniranja klasifikacijskog modela.

```
# Dodavanje oznake da se radi o antimikrobnim pept idima.
df_clean = add_label(df_clean, label= 1 )

# Prikaz prvih pet redaka.
df_clean.head()
```

	ID	COMPLEXITY	NAME	N TERMINUS	SEQUENCE	TERM
0	8	Monomer	NaN	C16	KVVVKWVVKVVK	
1	10	Monomer	NaN	NaN	LFIFFF	
2	11	Monomer	Cathelcidin-1, CATH-1, Fowlicidin-1	NaN	RVKRVWPLVIRTVIAGYNLYRAIKKK	

Opis rezultata

U ovom koraku svakom peptidu dodijeljena je oznaka klase pomoću funkcije `add_label(df_clean, label=1)`. Budući da skup podataka sadrži isključivo antimikrobni peptid, svim zapisima dodijeljena je vrijednost **1**, koja predstavlja pozitivnu klasu.

Nakon izvršavanja funkcije u skupu podataka dodan je novi stupac **label**. Prikaz prvih pet redaka potvrđuje da je oznaka uspješno dodana te da svi zapisi imaju vrijednost **1**. Ostali podaci, uključujući aminokiselinske sekvence (**SEQUENCE**) i ostala svojstva peptida, ostali su nepromijenjeni.

Dodavanje klasa klasa važan je korak u pripremi podataka za klasifikaciju jer modeli strojnog učenja uče povezivanjem ulaznih značajki s pripadajućim oznakama klase. U ovom slučaju vrijednost **1** označava antimikrobni peptid, odnosno pozitivnu klasu.

U sljedećoj fazi projekta ovom će skupu podataka biti pridruženi negativni primjeri s oznakom **0**. Time će biti formiran uravnotežen skup podataka koji će se koristiti za treniranje i evaluaciju klasifikacijskog modela.

7. Spremanje obrađenog AMP skupa

Obrađeni skup antimikrobnih peptida sprema se u novu CSV datoteku kako bi se mogao koristiti u sljedećim fazama projekta, poput izdvajanja značajki, treniranja i evaluacije modela strojnog učenja.

```
# Spremanje obrađenog skupa podataka.  
save_processed_dataset(  
    df_clean, "/content/amp_clean.csv" )
```

```
import os    # Modul za rad s datotekama i direktorijima.
```

```
# Ispis sadržaja mape /content radi provjere je li datoteka uspješno spremljena  
os.listdir( "/sadržaj" )
```

```
['.config',  
'non_amp_clean.csv',  
'__pycache__',  
'peptidi.csv',  
'features.py',  
'skup_podataka.csv',  
'AMPBenchmark_public.fasta',  
'peptidi-fasta.txt',  
'amp_clean.csv',  
'data_utils.py',  
'značajke.csv',  
'primjer_podataka']
```

Opis rezultata

U ovom koraku obrađeni skup antimikrobnih peptida spremljen je u novu CSV datoteku pod nazivom **amp_clean.csv**. Datoteka sadrži očišćene podatke i spremna je za daljnje faze projekta.

Nakon spremanja pomoću naredbe `os.listdir("/content")` provjeren je sadržaj direktorija **/content**. Budući da se u ispisu nalazi datoteka **amp_clean.csv**, potvrđeno je da je spremanje uspješno provedeno.

Datoteka **amp_clean.csv** koristit će se u sljedećim fazama projekta za izdvajanje značajki, treniranje i evaluaciju modela strojnog učenja.

8. Provjera nedostajućih vrijednosti

U ovom koraku provjerava se broj nedostajućih vrijednosti u svim stupcima skupa podataka kako bi se procijenio njihov utjecaj na daljnju analizu.

```
# Provjera nedostajućih vrijednosti.  
check_missing_values(df)
```

	0
ID	0
COMPLEXITY	0
NAME	309
N TERMINUS	1857
SEQUENCE	0
C TERMINUS	1075
SYNTHESIS TYPE	0
TARGET GROUP	17
TARGET OBJECT	111

dtype: int64

Opis rezultata

U ovom koraku provjerene su nedostajuće vrijednosti u svim stupcima skupa podataka. Cilj provjere bio je utvrditi u kojim stupcima postoje nedostajući podaci (NaN) te procijeniti njihov mogući utjecaj na daljnju obradu i analizu.

Rezultati pokazuju da stupci **ID**, **COMPLEXITY**, **SEQUENCE** i **SYNTHESIS TYPE** ne sadrže nedostajuće vrijednosti. Nedostajući podaci prisutni su u stupcima **NAME** (309), **N TERMINUS** (1857), **C TERMINUS** (1075), **TARGET GROUP** (17) i **TARGET OBJECT** (111).

Posebno je važno da stupac **SEQUENCE**, koji predstavlja osnovu za izdvajanje bioinformatičkih značajki i treniranje modela, nema nijednu nedostajuću vrijednost. Budući da se nedostajući podaci nalaze uglavnom u opisnim atributima koji se ne koriste za izgradnju modela, nije potrebno ukloniti dodatne zapise niti nadomjestiti nedostajuće vrijednosti.

9. Izvještaj o čišćenju podataka

Prikazuje se koliko je zapisa uklonjeno tijekom postupka čišćenja te koliko ih je ostalo za daljnju obradu.

```
# Broj zapisa prije čišćenja.
before = len(df_monomer)

# Broj zapisa nakon čišćenja.
after = len(df_clean)

print("=" * 40)
print("IZVJEŠTAJ O ČIŠĆENJU PODATAKA")
print("=" * 40)
print(f"Monomeri prije čišćenja : {before}")
print(f"Monomeri nakon čišćenja : {after}")
print(f"Uklonjeno zapisa          : {before - after}")
print("=" * 40)
```

```
=====
IZVJEŠTAJ O ČIŠĆENJU PODATAKA
=====
Monomeri prije čišćenja : 1976
Monomeri nakon čišćenja : 1672
Uklonjeni zapis: 304
=====
```

Opis rezultata

Rezultati pokazuju da je prije čišćenja skup podataka sadržavao **1976 monomernih peptida**, dok je nakon čišćenja ostalo **1672** zapisa. To znači da su tijekom postupka čišćenja uklonjena **304** zapisa koji nisu zadovoljili zadane kriterije kvalitete, poput neispravnih sekvenci, nestandardnih aminokiselina ili dupliciranih zapisa.

Dobiveni izvještaj pruža jasan pregled učinka postupka čišćenja te potvrđuje da je završni skup podataka na način, ali kvalitetniji i prikladniji za daljnju analizu.

Uklanjanjem nekvalitetnih zapisa povećana je pouzdanost skupa podataka koji će se koristiti za izdvajanje značajki te za treniranje i evaluaciju klasifikacijskog modela.

10. Izračun duljine sekvenci

Izračunava se duljina svake peptidne sekvence te se prikazuju osnovne statističke mjere.

```
# Dodavanje stupca s duljinom svake peptidne sekvence.
df_clean["sequence_length"] = df_clean["SEQUENCE"].apply(len)

# Prikaz osnovne statistike duljina sekvenci.
df_clean["sequence_length"].describe()
```


sequence_length	
count	1672.000000
mean	21.490431
std	12.771008
min	1.000000
25%	13.000000
50%	19.000000
75%	26.000000
max	119.000000

dtype: float64

Opis rezultata

U ovom koraku za svaku peptidnu sekvencu izračunata je njezina duljina te je dodan novi stupanj `sequence_length`. Nakon toga prikazane su osnovne deskriptivne statistike koje daju pregled raspodjele duljine sekvenci u skupu podataka.

Rezultati pokazuju da skup sadrži **1672 peptidne sekvence**. Prosječna duljina sekvence iznosi **21,49 aminokiselina**, dok je medijan **19 aminokiselina**, što znači da polovica sekvence ima duljinu manju ili jednaku 19 aminokiselina. Najkraća sekvenca sadrži **1 aminokiselinu**, a najdulja **119 aminokiselina**.

Vrijednosti kvartila pokazuju da **25 % sekvenci ima najviše 13 aminokiselina**, dok **75 % sekvenci ima najviše 26 aminokiselina**. Standardna devijacija iznosi **12,77**, što ukazuje na određenu varijabilnost duljine sekvence u skupu podataka.

Duljina sekvenca predstavlja jednu od bioinformatičkih značajki koja će se koristiti u daljnjim fazama projekta prilikom treniranja modela strojnog učenja.

11. Histogram duljina sekvenci

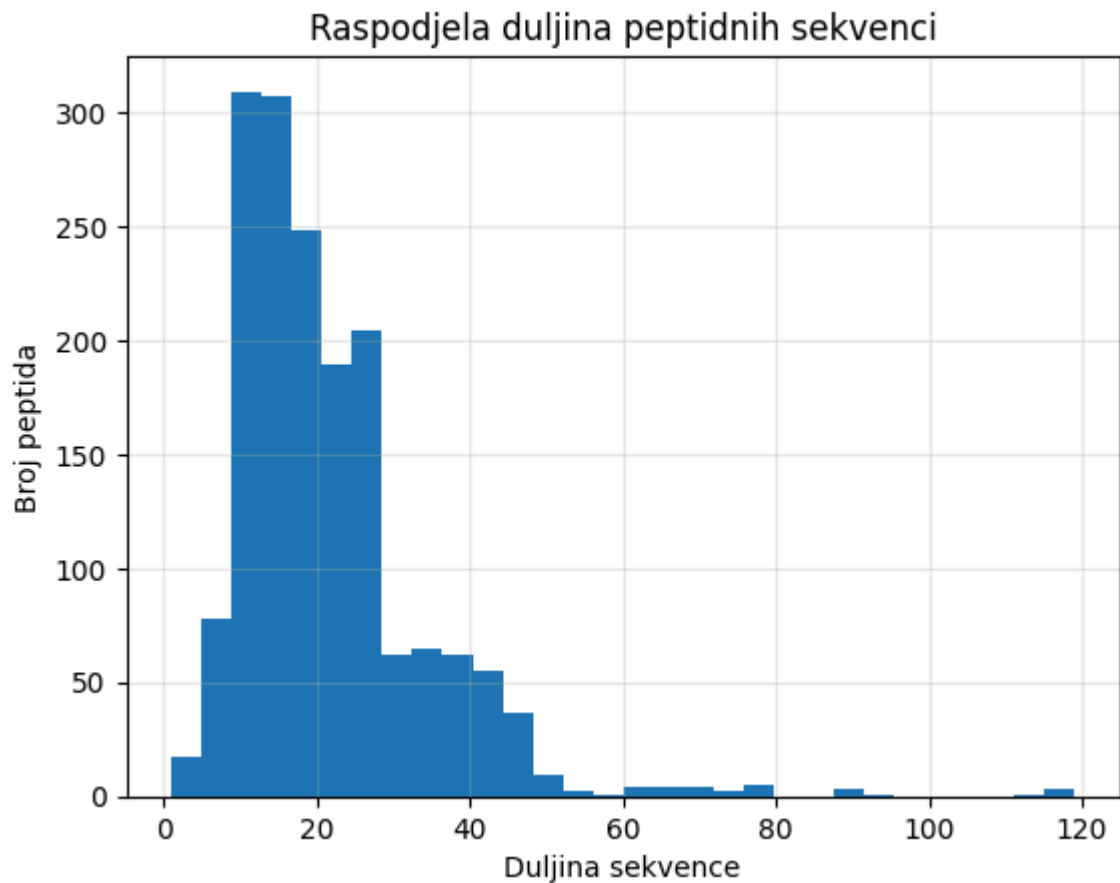
U ovom koraku izrađuje se histogram kojim se prikazuje raspodjela duljine peptidnih sekvenci u analiziranom skupu podataka

```
# Uvoz biblioteke za izradu grafova.
import matplotlib.pyplot as plt

# Izrada histograma duljina peptidnih sekvenci.
plt.hist(df_clean["sequence_length"], bins=30)

# Dodavanje naslova grafa.
plt.title("Raspodjela duljina peptidnih sekvenci")
```

```
# Oznaka x-osi.  
plt.xlabel("Duljina sekvence")  
  
# Oznaka y-osi.  
plt.ylabel("Broj peptida")  
  
# Dodavanje mreže radi lakšeg očitavanja vrijednosti.  
plt.grid(alpha=0.3)
```



Opis rezultata

Na histogramu je prikazana raspodjela duljine peptidnih sekvenci u skupu podataka. Na x-osi prikazana je duljina sekvence, a na y-osi broj peptida.

Graf pokazuje da je najveći broj peptidnih sekvenci kraće duljine, dok se broj sekvenci postupno smanjuje kako duljina raste. Vrlo duge sekvence pojavljuju se rijetko, što ukazuje da većinu skupa podataka čine kraći peptidi.

```
from google.colab import drive  
drive.mount('/content/drive')
```

12. Sažetak obrade podataka

Prikazuje se broj zapisa nakon svake faze obrade podataka kako bi se jasno vidio tijek pripreme skupa podataka.

```
#Broj zapisa kroz cjevovod
summary = {
    "Korak": [
        "Početni skup",
        "Monomeri",
        "Očišćeni skup"
    ],
    "Broj zapisa": [
        len(df),
        len(df_monomer),
        len(df_clean)
    ]
}

import pandas as pd

pd.DataFrame(summary)
```

	Korak	Broj zapisa
0	Početni skup	2000
1	Monomeri	1976
2	Očišćeni skup	1672

Opis rezultata

Rezultati pokazuju da je izvorni skup podataka sadržavao **2000 zapisa**. Nakon izdvajanja monomernih peptida broj zapisa smanjen je **1976. godine**, što znači da su uklonjena **24 zapisa** koja su pripadala kategorijama multi-peptida i multimeri.

Nakon provedenog čišćenja sekvenci, uklanjanja neispravnih zapisa i duplikata, konačni skup podataka sadrži **1672 zapisa**. Time je dobiven kvalitetniji i pouzdaniji skup podataka koji će se koristiti u sljedećim fazama projekta za izdvajanje značajki (feature engineering), treniranje i evaluaciju klasifikacijskog modela.

Sažetak jasno prikazuje tijek pripreme podataka, od izvornog skupa do konačno obrađenog skupa. Provedenim filtriranjem i čišćenjem uklonjeni su zapisi koji nisu bili prikladni za daljnju analizu, čime je osigurana kvalitetna osnova za nastavak rada.

S Priprema negativnog (non-AMP) skupa podataka

Kako bi model mogao razlikovati antimikrobne od neantimikrobnih peptida, potrebno je pripremiti i **negativnu klasu (non-AMP)**. Negativni primjeri preuzeti su iz javno dostupnog skupa podataka **AMPBenchmark**, iz kojeg se izdvajaju sekvence označene kao **AMP=0**. Nakon toga prolazi isti postupak obrade kao i pozitivni primjeri.

13. Uvoz funkcija za rad s FASTA datotekama

U ovom koraku uvoze se funkcije potrebne za učitavanje i obradu AMPBenchmark FASTA datoteke te izdvajanje negativnih primjera (AMP = 0).

```
# Uvoz funkcija za rad s CSV datotekama.
from data_utils import (
    load_dataset,
    check_required_columns,
    print_dataset_info,
    check_missing_values,
    filter_monomers,
    clean_sequences,
    add_label,
    save_processed_dataset,
)

# Uvoz funkcija za rad s AMPBenchmark FASTA datotekama.
from data_utils import (
    load_ampbenchmark_fasta,
    extract_ampbenchmark_by_label,
)
```

Opis rezultata

U ovom koraku uvezene su funkcije potrebne za rad s **AMPBenchmark FASTA** datotekom. Uvezene funkcije omogućuju učitavanje FASTA datoteke te izdvajanje peptidnih sekvenci prema oznakama klase (**AMP = 0** ili **AMP = 1**).

Za potrebe ovog projekta koristit će se sekvence označene kao **AMP = 0**, koje predstavljaju **negativne (non-AMP) primjere**. One će se u sljedećim koracima spojiti s pozitivnim primjerima iz baze **DBAASP**, čime će se formirati konačan skup podataka za treniranje i evaluaciju klasifikacijskog modela.

14. Učitavanje FASTA datoteke

Učitava se AMPBenchmark FASTA datoteka koja sadrži velik broj peptidnih sekvenci.

```
# Putanja do AMPBenchmark FASTA datoteke.
fasta_path = "AMPBenchmark_public.fasta"

# Učitavanje FASTA datoteke.
ampbenchmark_df = load_ampbenchmark_fasta(fasta_path)

# Prikaz prvih nekoliko redaka.
ampbenchmark_df.head()
```

	ID	SEQUENCE
0	DBAASP_10018_AMP=1_rep1	WMLKKFRGMF
1	DBAASP_3217_AMP=1_rep1	AWRFKNIRK
2	DBAASP_11239_AMP=1_rep1	LIPFPFP
3	DBAASP_9251_AMP=1_rep1	PLLKKLLKKP
4	DBAASP_1175_AMP=1_rep1	QKKIRARLSA

Opis rezultata

U ovom koraku učitana je **AMPBenchmark FASTA** datoteka pomoću funkcije `load_ampbenchmark_fasta()`. Nakon učitavanja prikazan je prvi pet zapisa kako bi se provjerilo je li datoteka uspješno učitana.

Prikazani podaci sadrže dva osnovna stupca: **ID**, koji predstavlja jedinstveni identifikator peptida, i **SEQUENCE**, u kojem se nalazi aminokiselinska sekvenca peptida.

Iz identifikatora je vidljivo da sadrži oznaku klase, npr. **AMP=1**, koja označava antimikrobni peptid. U sljedećem koraku iz ove datoteke izdvojiti će se sekvence označene kao **AMP=0**, koje predstavljaju negativne (non-AMP) primjere i koristiti će se za izgradnju klasifikacijskog modela.

```
# Broj svih FASTA zapisa.
print(f"Ukupan broj FASTA zapisa: {len(ampbenchmark_df)}")
```

Ukupan broj FASTA zapisa: 128479

```
# Izdvajanje negativnih (non-AMP) sekvenci.
non_amp_df = extract_ampbenchmark_by_label(
    ampbenchmark_df,
    label=0
)

# Prikaz prvih nekoliko redaka.
non_amp_df.head()
```

ID		
0	Seq2_sampling_method=iAMP- 2L_AMP=0_rep1	MNVSNAGIDSNDLAKRGESLIRKSTNRYLTTVRIAFF
1	Seq3_sampling_method=iAMP- 2L_AMP=0_rep1	MAKKSMIARDVKRKKIVERYAAKRAALMEAFNAAKDP
2	Seq22_sampling_method=iAMP- 2L_AMP=0_rep1	KREHEKEVLQKAIEENNNFSKMAEEKLTTKMEAIKENF

```
print(f"Broj negativnih primjera: {len(non_amp_df)}")
```

```
Broj negativnih primjera: 123284
```

Opis rezultata

U ovom koraku najprije je određen ukupni broj zapisa u **AMPBenchmark FASTA** datoteci. Rezultat pokazuje da datoteka sadrži **128 479 peptidnih sekvenci**, što potvrđuje da se radi o velikom skupu podataka.

Zatim su pomoću funkcije `extract_ampbenchmark_by_label()` izdvojene sekvence s oznakom **AMP = 0**, odnosno peptidi koji nisu antimikrobni. Izdvojena su ukupno **123 284 negativna primjera**.

Prikaz prvih nekoliko redaka potvrđuje da izdvojeni skup sadrži stupce **ID**, **SEQUENCE** i **label**, pri čemu svi zapisi imaju vrijednost **label = 0**, što potvrđuje da pripadaju negativnoj klasi (non-AMP).

Dobiveni skup negativnih primjera koristit će se u nastavku projekta zajedno s pozitivnim primjerima iz baze **DBAASP** za izgradnju i evaluaciju klasifikacijskog modela.

16. Nasumično uzorkovanje negativnih primjera

Nasumično se odabire jednak broj negativnih primjera kao pozitivnih kako bi skup podataka bio uravnotežen.

```
# Nasumično odabiremo isti broj non-AMP sekvenci kao AMP sekvenci.
non_amp_sample = non_amp_df.sample(
    n=1672,
    random_state=42
)

# Resetiramo indeks.
non_amp_sample = non_amp_sample.reset_index(drop=True)

# Provjera dimenzija.
```

```
non_amp_sample.shape
```

```
(1672., 3)
```

Opis rezultata

U ovom koraku provedeno je nasumično uzorkovanje negativnih primjera pomoću funkcije `sample()`. Budući da očišćeni skup pozitivnih primjera sadrži **1672 antimikrobna peptida**, iz skupa negativnih primjera odabran je isti broj (**1672**) kako bi obje klase bile jednako zastupljene. Da su korišteni svi negativni primjeri (**123 284**), model bi bio pristran prema negativnoj klasi i mogao bi dati lošije rezultate za prepoznavanje antimikrobnih peptida.

Za uzorkovanje se koristi parametar `random_state=42`, čime je osigurano da se pri svakom pokretanju programa odaberu isti primjeri. Nakon toga primijenjena je funkcija `reset_index(drop=True)` kako bi se ponovno numerirali indeksi dobivenog skupa podataka.

Rezultat naredbe `non_amp_sample.shape` iznosi **(1672, 3)**, što znači da uzorkovani skup sadrži **1672 zapisa** i **3 stupca** (**ID**, **SEQUENCE** i **label**).

Na ovaj način dobiven je **uravnotežen skup podataka** s jednakim brojem pozitivnih i negativnih primjera. Takav skup omogućuje pravednije treniranje klasifikacijskog modela i pouzdanost procjene njegove uspješnosti.

```
# Prikaz prvih nekoliko redaka.
non_amp_sample.head()
```

	ID	
0	Seq1734_sampling_method=Wang-et-al_AMP=0_rep3	REL
1	Seq1021_sampling_method=AMAP_AMP=0_rep3	AQHNENNKETVMMPIIQLNA
2	Seq747_sampling_method=Witten&Witten_AMP=0_rep3	I
3	Seq422_sampling_method=Wang-et-al_AMP=0_rep1	LEDQAPRRI
4	Seq1188_sampling_method=Gabere&Noble_AMP=0_rep4	

Opis rezultata

Nakon provedenog nasumičnog uzorkovanja prikazano je prvih pet redaka novog skupa podataka pomoću funkcije `non_amp_sample.head()`. Time je provjerena ispravnost izdvajanja i priprema negativnih primjera za daljnju analizu.

Iz prikaza je vidljivo da skup podataka sadrži stupce **ID** , **SEQUENCE** i **label** .

Identifikatori sekvenci sadrže oznaku **AMP = 0** , dok stupac **label** za sve prikazane zapise ima vrijednost **0** , što potvrđuje da se u skupu nalaze isključivo neantimikrobni peptidi.

Također se može uočiti da uzorkovane sekvence imaju različite identifikatore i različite duljine, što potvrđuje da je nasumično uzorkovanje odabralo raznolike negativne primjere iz izvornog AMPBenchmark skupa podataka.

Prikaz prvih nekoliko redaka predstavlja završnu provjeru ispravnosti pripreme negativnih primjera prije njihova spajanja s pozitivnim primjerima. Time je dobiven uravnotežen skup negativnih podataka spreman za formiranje konačnog skupa za treniranje klasifikacijskog modela.

17. Spremanje negativnog (non-AMP) skupa podataka

Obrađeni negativni skup podataka sprema se u zasebnu CSV datoteku.

```
# Spremanje negativnog skupa.  
save_processed_dataset(  
    non_amp_sample,  
    "/content/non_amp_clean.csv"  
)
```

Opis rezultata

U ovom koraku obrađeni skup negativnih primjera spremljen je u zasebnu CSV datoteku pomoću funkcije `save_processed_dataset()`. Spremljena datoteka nazvana je **non_amp_clean.csv** i sadrži nasumično odabrane neantimikrobne peptide s oznakom klase **0** , spremne za daljnju obradu.

Spremanjem obrađenog skupa podataka omogućeno je njegovo ponovno korištenje u sljedećim fazama projekta bez potrebe za ponovnim učitavanjem, filtriranjem i uzorkovanjem podataka. Time se pojednostavljuje daljnji rad i osigurava reproduktivnost postupka.

Dobivena datoteka **non_amp_clean.csv** predstavlja konačni skup negativnih primjera koji će se u nastavku projekta spojiti s pozitivnim skupom **amp_clean.csv** kako bi se formirao uravnotežen skup podataka za treniranje klasifikacijskog modela.

18. Učitavanje i spajanje obrađenih skupova podataka

Učitavaju se prethodno spremljeni pozitivni i negativni skupovi podataka radi njihova spajanja u jedinstveni skup, nakon čega se redoslijed zapisa

nasumično miješa kako bi podaci bili prikladno pripremljeni za treniranje klasifikacijskog modela.

```
import pandas as pd
```

```
amp_clean = pd.read_csv("/content/amp_clean.csv")
```

```
# Učitavanje očišćenog AMP skupa.
amp_clean = pd.read_csv("/content/amp_clean.csv")

# Učitavanje očišćenog non-AMP skupa.
non_amp_clean = pd.read_csv("/content/non_amp_clean.csv")

# Zadržavamo samo stupce koji su potrebni za nastavak projekta.
amp_clean = amp_clean[["ID", "SEQUENCE", "label"]]
non_amp_clean = non_amp_clean[["ID", "SEQUENCE", "label"]]

# Spajamo pozitivnu i negativnu klasu u jedan skup podataka.
dataset = pd.concat([amp_clean, non_amp_clean], ignore_index=True)

# Miješamo redoslijed redaka radi neutralnosti pri kasnijem modeliranju.
dataset = dataset.sample(frac=1, random_state=42).reset_index(drop=True)

# Provjera dimenzija konačnog skupa.
dataset.shape
```

```
(3344, 3)
```

Opis rezultata

U ovom koraku učitavaju se dva prethodno pripremljena skupa podataka: skup antimikrobnih peptida (**AMP**) i skup neantimikrobnih peptida (**non-AMP**). Iz oba skupa zadržani su samo stupci **ID** , **SEQUENCE** i **label** , budući da su oni potrebni za daljnju analizu i treniranje klasifikacijskog modela.

Nakon učitavanja oba su skupa spojena u jedinstveni DataFrame pomoću funkcije `pd.concat()` , čime je formiran zajednički skup podataka koji sadrži i pozitivne (**label = 1**) i negativne (**label = 0**) primjere.

Kako bi se izbjegao utjecaj izvornog redoslijeda zapisa na treniranje modela, svi su zapisi nasumično promiješani primjenom funkcije `sample(frac=1, random_state=42)` , a zatim su indeksi ponovno numerirani pomoću funkcije `reset_index(drop=True)` . Vrijeme je osigurano da su pozitivni i negativni primjeri ravnomjerno raspoređeni u konačnom skupu podataka.

Rezultat naredbe `dataset.shape` iznosi **(3344, 3)**, što znači da konačni skup podataka sadrži **3344 zapisa** i **3 stupca** (`ID`, `SEQUENCE` i `label`). Budući da skup uključuje **1672 pozitivna** i **1672 negativna** primjera, obje klase jednako su zastupljene, čime je dobiven uravnotežen skup podataka prikladan za treniranje i evaluaciju klasifikacijskog modela.

20. Provjera raspodjele klasa u konačnom skupu podataka

Provjerava se broj uzoraka u svakoj klasi kako bi se potvrdilo da je skup podataka uravnotežen.

```
# Provjera raspodjele klasa.  
dataset["label"].value_counts()
```

	count
label	
1	1672
0	1672

dtype: int64

Opis rezultata

U ovom koraku provedena je provjera raspodjele klasa u konačnom skupu podataka pomoću funkcije `value_counts()`. Cilj provjere bio je utvrditi broj pozitivnih i negativnih primjera te potvrditi da je skup podataka uravnotežen prije treniranja klasifikacijskog modela.

Rezultati pokazuju da skup podataka sadrži **1672 antimikrobna peptida (label = 1)** i **1672 neantimikrobna peptida (label = 0)**. Jednaka zastupljenost obje klase potvrđuje da je postupak pripreme podataka uspješno proveden te da nije došlo do neravnoteže između pozitivnih i negativnih primjera.

Uravnotežena raspodjela klasa važna je za treniranje modela strojnog učenja jer smanjuje mogućnost pristranosti prema jednoj klasi i omogućuje pouzdanu procjenu uspješnosti klasifikacije. Na temelju dobivenih rezultata može se zaključiti da je konačni skup podataka pravilno pripremljen i spreman za sljedeće faze projekta.

21. Spremanje konačnog skupa

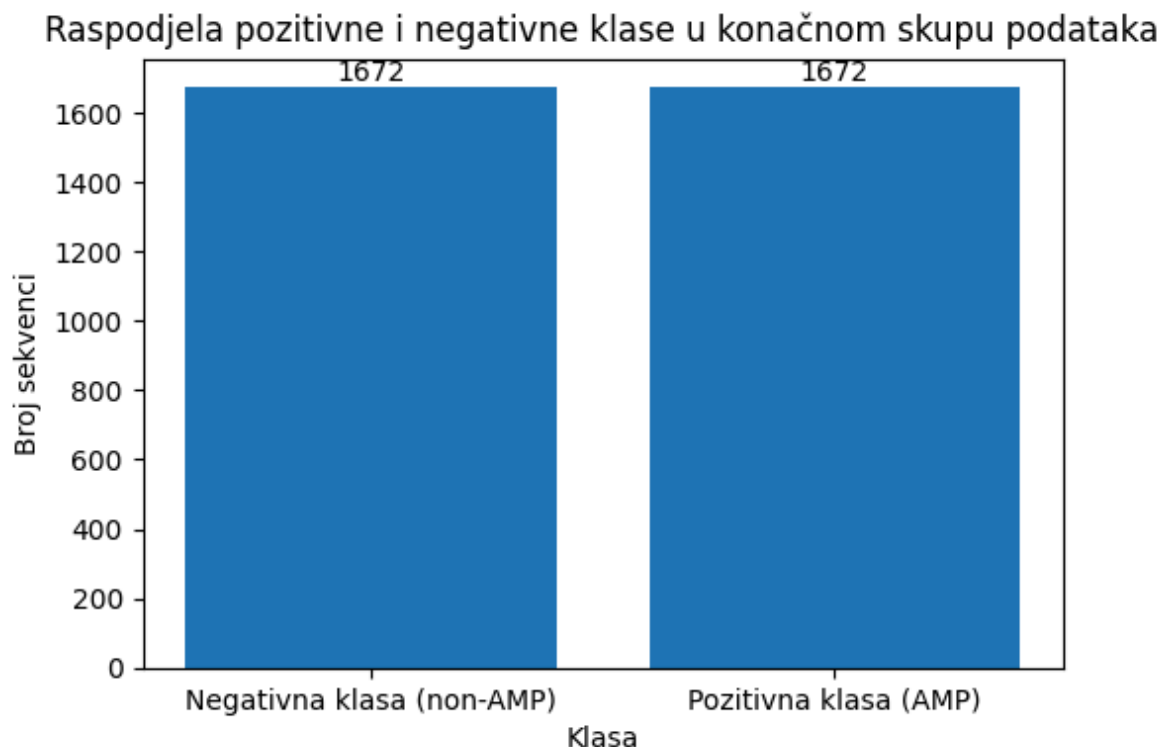
Konačni skup podataka sprema se za korištenje u fazi ekstrakcije značajki i modela treniranja.

```
save_processed_dataset(  
    dataset,  
    "/content/dataset.csv"  
)
```

22. Grafički prikaz raspodjele pozitivne i negativne klase

Prikazuje se graf raspodjele pozitivne i negativne klase kojim se potvrđuje jednak broj uzoraka u obje klase.

```
# Uvoz biblioteke za crtanje grafova.  
import matplotlib.pyplot as plt  
  
# Izračun broja uzoraka u svakoj klasi.  
class_counts = dataset["label"].value_counts().sort_index()  
  
# Crtanje stupčastog grafa.  
plt.figure(figsize=(6, 4))  
  
plt.bar(  
    ["Negativna klasa (non-AMP)", "Pozitivna klasa (AMP)"],  
    class_counts.values  
)  
  
# Dodavanje naslova.  
plt.title("Raspodjela pozitivne i negativne klase u konačnom skupu podataka")  
  
# Naziv x osi.  
plt.xlabel("Klasa")  
  
# Naziv y osi.  
plt.ylabel("Broj sekvenci")  
  
# Dodavanje vrijednosti iznad stupaca.  
for i, value in enumerate(class_counts.values):  
    plt.text(i, value + 20, str(value), ha="center")  
  
# Prikaz grafa.  
plt.show()
```



Opis rezultata

U ovom koraku izrađen je stupčasti grafikon koji prikazuje raspodjelu pozitivne i negativne klase u konačnom skupu podataka. Broj uzoraka u svakoj klasi izračunat je pomoću funkcija `value_counts()`, a rezultati su grafički prikazani korištenjem funkcija `plt.bar()`.

Na vodoravnoj osi prikazane su dvije klase: **negativna klasa (non-AMP)** i **pozitivna klasa (AMP)**, dok okomito prikazuje broj sekvenci u svakoj klasi. Iznad svakog stupca prikazan je točan broj uzoraka radi lakšeg očitavanja rezultata.

Iz grafa je vidljivo da obje klase sadrže **1672 sekvence**, što potvrđuje da je konačni skup podataka potpuno uravnotežen. Jednaka visina oba stupca pokazuje da su pozitivni i negativni primjeri jednako zastupljeni te da tijekom priprema podataka nije došlo do neravnomjerne raspodjele klasa.

Graf predstavlja vizualnu potvrdu prethodno dobivenih brojčanih rezultata te pokazuje da je konačni skup podataka pravilno pripremljen za treniranje i evaluaciju klasifikacijskog modela strojnog učenja.

23. Završni ispis

Opisuju se osnovne informacije o konačnom skupu podataka koje služe kao završna provjera uspješnosti pripreme podataka.

```
# Prikaz osnovnih informacija o konačnom skupu.
print("=" * 40)
print("FINAL DATASET")
print("=" * 40)

print(f"Ukupan broj sekvenci : {len(dataset)}")
print(f"AMP : {(dataset['label'] == 1).sum()}")
print(f"non-AMP : {(dataset['label'] == 0).sum()}")

print("=" * 40)=====
ZAVRŠNI SCUP PODATAKA
=====
Ukupan broj sekvenci: 3344
AMP: 1672
ne-AMP: 1672
=====
```

Opis rezultata

U završnom koraku ispisane su osnovne informacije o konačnom skupu podataka. Cilj ovog ispisa bio je potvrditi da su svi prethodni koraci pripreme podataka uspješno provedeni te da je dobiven skup podataka spreman za daljnje faze projekta.

Rezultati pokazuju da konačni skup podataka sadrži ukupno **3344 peptidne sekvence**, od čega je **1672 antimikrobnih peptida (AMP)** i **1672 neantimikrobnih peptida (non-AMP)**. Jednaka zastupljenost obje klase potvrđuje da je skup podataka potpuno uravnotežen i prikladan za treniranje klasifikacijskog modela.

Završni ispis predstavlja posljednju provjeru uspješnosti pripreme podataka te potvrđuje da je konačni skup podataka pravilno formiran i spreman za sljedeću fazu projekta.

Zaključak

U ovom dijelu projekta provedena je priprema podataka za klasifikaciju antimikrobnih peptida. Pozitivni skup podataka preuzet je iz baze **DBAASP**, dok je negativni skup izdvojen iz javno dostupnog skupa podataka **AMPBenchmark**.

Tijekom priprema podataka provedeno je filtriranje monomernih peptida, provjera i čišćenje sekvenci te uklanjanje neispravnih i dupliciranih zapisa. Pozitivnim primjerima dodatno je izvršeno čišćenje i izdvojen skup podataka iz baze **AMPBenchmark** koji sadrži negativne primjere.